

AMMAN ARAB UNIVERSITY

جامعة عمان العربية



كلية تكنولوجيا المعلومات
College of Information Technology

Optimizing ChaCha20 for Secure Decentralized Networks

Students:

Rawand amer abusharkh

ID: 202110053

Abdullah saad alsaad

ID: 202111089

Reem wesam abo asfar

ID: 202110114

Supervisor:

Dr. Saad Ismail

Co-supervisor: Dr. Kamal Alieyan

A project report submitted in partial fulfillment of the requirements

for B.Sc. degree in cybersecurity

Amman – Jordan 2024/2025

Acknowledgment

First and foremost, I would like to express my sincere gratitude to Dr. Saad Ismail & Dr. Kamal Alieyan for his continuous support, guidance, and encouragement throughout the duration of this graduation project. His insightful feedback and valuable suggestions have been instrumental in shaping the direction and quality of my work.

I would also like to extend my thanks to the faculty and staff of the Cybersecurity Department at Amman Arab University for providing a solid foundation of knowledge and for fostering an environment conducive to learning and research.

Special thanks to my family and friends for their unconditional support, motivation, and patience during the challenging phases of this project. Your belief in me has been a constant source of strength.

Finally, I would like to thank everyone who contributed in any way to the completion of this project. Your assistance and encouragement are deeply appreciated

Abstract

The rapid evolution of cryptography has become pivotal in securing digital communication, particularly in sensitive and high-risk environments such as the Dark Web and Deep Web. These networks demand robust encryption algorithms to ensure data confidentiality, integrity, and resistance against sophisticated cyberattacks. This project focuses on the ChaCha20 algorithm, a modern stream cipher known for its high performance and strong security. The research aims to evaluate and optimize the performance of ChaCha20 within resource-constrained environments, such as mobile devices and IoT systems, and integrate it into security protocols commonly used in the Dark Web and Deep Web. Through an in-depth analysis of ChaCha20's design, implementation, and testing across various platforms, the project explores ways to enhance its efficiency while maintaining its cryptographic strength. Key outcomes include the development of optimization techniques to improve the algorithm's speed and resource utilization, integration into secure protocols like TOR, and a comprehensive assessment of its resistance to emerging cryptographic threats. This work contributes to advancing encryption technologies for enhanced security in the increasingly challenging landscape of private and decentralized networks.

Table of Contents

Acknowledgment.....	
Abstract	II
List of Contents	III
List of Tables.....	V
List of Figures	VI
Chapter 1	1
Introduction	1
1.1 Project Background.....	1
1.2 Problem Statement	2
1.3 Project Objectives	3
1.4 Significant of Project	3
1.5 Research strategy (Framework).....	4
1.6 Project Motivation	6
1.7 Tools and Technologies Used	7
1.8 Scope of Research	7
1.9 Project Plan.....	8
1.10 Project outline	9
Chapter 2	10
Literature Review	10
2.1 Overview	10
2.2 Critical Evaluation (Gap Analysis)	10
2.3 Related work	11
2.3.1 Discussions of What Has Been Done Along the Line of the Problem	13
2.3.2 Issues and Challenges Faced by Other Projects	14
2.3.3 Critical Evaluation (Gap Analysis)	14
2.4 Summary	15

Chapter 3	16
Methodology	16
3.1 System Analysis	16
3.1.1 Requirement Elicitation Techniques	19
3.1.2 Specific Requirements	20
3.1.2.2 Functional Requirements.....	23
3.1.2.3 Design Constructs	24
3.1.3 Non-Functional Requirements	26
3.1.3.1 Data Flow Diagrams (DFDs)	27
3.1.3.2 System Design	30
 Chapter 4	 33
System Implementation.....	33
4.1 Programming Language and Libraries.....	33
4.1.1 Justification for Using Python.....	33
4.1.2 Justification for Using Cryptography Library.....	33
4.1.3 Justification for Using tkinter.....	34
4.2 System Workflow and Components.....	34
4.2.1 File Benchmarking Logic.....	34
4.2.2 Pseudocode Representation.....	35
4.2.3 GUI Components Overview.....	35

Chapter 5	36
System Evaluation and Testing.....	36
5.1 Testing Plan.....	36
5.1.1 Functional Testing.....	36
5.1.2 GUI Testing.....	37
5.1.3 Performance Testing.....	37
5.2.1 File-Based Testing Scenarios.....	37
5.2.2 Decryption Validation.....	38
5.2.3 Performance Consistency.....	38
5.2.4 Performance Screenshots.....	38
 Chapter 6	 40
Conclusions.....	40
6.1 Conclusion.....	40
6.2 Future Work.....	41
References.....	43

List of tables

Table 2.1: Summary of the performance characteristics for various hardware cypher....	12
Table 2.2: Issues and challenges faced by the other project.....	14
Table 2.3: Compares AES and ChaCha20.....	15
Table 3.1:Fields of ChaCha20 initial state.....	17
Table3.2: ChaCha20 even round input words.....	19
Table3.3: ChaCha20 odd round input words.....	19
Table 3.4:Types of Interfaces and Their Roles in ChaCha20 Encryption.....	22
Table 5.1: File-Based Testing Scenarios.....	37

List of Figures

Figure 1.1: The Framework.....	5
Figure 1.2: Gantt chart.....	8
Figure 3.1: Graphical illustration of ChaCha20 algorithm.....	16
Figure 3.2: Graphical illustration of ChaCha20 round.....	18
Figure 3.3: DFD Level 0.....	28
Figure 3.4: DFD Level 1.....	29
Figure 3.5: DFD Level 2.....	30
Figure 3.6: Flow Diagram.....	31
Figure 3.7: Use Cases diagram.....	31
Figure 3.8: Activity Diagram.....	33
Figure 4.1 : Pseudocode Representation.....	35
Figure 5.1: shows the results before optimization.....	39
Figure 5.2: presents the results after optimization.....	39

CHAPTER 1

INTRODUCTION

1.1 Project Background

The internet can be broadly categorized into three layers: the Surface Web, the Deep Web, and the Dark Web. While the Surface Web represents the publicly accessible portion indexed by traditional search engines, the Deep Web and Dark Web remain hidden from standard browsing tools, often misunderstood and shrouded in mystery.

The Deep Web refers to parts of the internet that are not indexed by search engines. This includes databases, academic journals, medical records, private forums, and internal company systems. These resources require specific permissions or credentials to access, contributing to their invisibility from public search engines. Contrary to popular belief, the Deep Web forms a significant portion of the internet and serves legitimate functions, such as protecting sensitive data and facilitating confidential communication.

The Dark Web, a subset of the Deep Web, is intentionally hidden and requires specialized tools like Tor (The Onion Router) or I2P (Invisible Internet Project) for access. While the Dark Web is often associated with illegal activities, such as black markets, illicit trade, and hacking forums, it also serves as a sanctuary for whistleblowers, journalists, and individuals seeking to bypass censorship. This dual nature creates a complex environment where privacy and anonymity are paramount. [1]

Despite its benefits, the Dark Web introduces significant security challenges. Malicious actors exploit the anonymity it provides to conduct illegal operations, increasing the risk of data breaches, identity theft, and the proliferation of harmful

software. Therefore, safeguarding communications and data transfers within these environments is critical.

Encryption plays a fundamental role in ensuring the security and integrity of data traversing the Deep Web and Dark Web. Robust encryption mechanisms prevent unauthorized access, ensuring that sensitive information remains confidential. Among the various encryption algorithms, ChaCha20 has emerged as a modern and efficient solution. Known for its speed, simplicity, and strong resistance to cryptographic attacks, ChaCha20 offers superior performance compared to older algorithms like AES (Advanced Encryption Standard).

Incorporating ChaCha20 into security protocols can significantly enhance privacy and data protection across the Deep Web and Dark Web. This project explores the application of the ChaCha20 encryption algorithm as a method to bolster security, mitigate risks, and improve overall network resilience in these concealed internet spaces. [1]

1.2 Problem Statement

Despite the proven efficiency and robust security of the ChaCha20 encryption algorithm, its practical application within sensitive environments such as the Deep Web and Dark Web faces several critical challenges. One of the primary issues is the need to enhance performance to ensure the algorithm operates effectively in resource-constrained networks, which are common in hidden services and decentralized systems. Additionally, as cyber threats continue to evolve, it is essential to strengthen the algorithm's security to counter advanced and emerging attacks that target encrypted communications. Another significant challenge lies in integrating ChaCha20 into existing protocols widely used across the Deep and Dark Web, ensuring seamless adoption without compromising efficiency or compatibility.

This project seeks to address these issues by exploring methods to optimize ChaCha20's implementation, improving both its speed and resilience in environments where privacy and security are paramount. By doing so, the

project aims to enhance the overall reliability of encryption practices in the Deep and Dark Web, aligning with the objective of creating more secure and efficient communication channels in these concealed areas of the internet. [1]

1.3 Project objectives

1. Analyze the Design and Functionality of the ChaCha20 Algorithm.
2. Evaluate the Performance of ChaCha20 in Deep Web and Dark Web Networks.
3. Optimize the Algorithm for Resource and Performance Constraints.
4. Enhance the Security of the Algorithm Against Modern Threats.
5. Integrate ChaCha20 into Security Protocols Used in the Deep Web and Dark Web. [2]

1.4 Significant of Project

Understanding the internal workings and design principles of ChaCha20 is fundamental to evaluating its strengths and limitations. The first objective involves a comprehensive study of how ChaCha20 operates, including its key generation process, block structure, and encryption/decryption procedures. By breaking down the algorithm step by step, the project aims to provide a clear insight into why ChaCha20 is considered secure and how it differs from traditional algorithms like AES. This analysis serves as the foundation for optimizing and integrating the algorithm into security frameworks.

The performance of encryption algorithms can vary depending on the environment in which they are deployed. This objective focuses on testing ChaCha20 within the unique conditions of Deep Web and Dark Web networks, where bandwidth limitations, latency, and decentralized infrastructures can affect efficiency. The goal is to measure how ChaCha20 handles data encryption in these environments and to identify potential bottlenecks or performance issues that may arise. [2]

One of the major challenges in hidden networks is the limited computational power and constrained resources of many nodes. This objective aims to refine ChaCha20's implementation to ensure minimal resource consumption without compromising its security. The project will explore techniques to accelerate the algorithm's execution, reduce overhead, and adapt it for devices with lower processing capabilities, making it a viable option for a broader range of applications within the Deep and Dark Web.

As cyber threats grow more sophisticated, even the most secure algorithms can become vulnerable to new attack vectors. This objective focuses on reinforcing ChaCha20's resilience against emerging threats, including quantum computing risks, side-channel attacks, and brute-force attempts. By identifying potential weaknesses and applying countermeasures, the project aims to future-proof ChaCha20, ensuring its continued effectiveness in safeguarding sensitive communications.

For ChaCha20 to have a practical impact, it must be incorporated into existing security protocols that govern anonymous communication channels. This objective involves embedding ChaCha20 into widely used protocols, such as the TOR (The Onion Router) network, which relies heavily on encryption to anonymize user activity. The project will explore methods to replace or complement existing encryption standards with ChaCha20, enhancing overall security while maintaining compatibility with existing infrastructure. [2]

1.5 Research strategy (Framework)

The project will adhere to a structured research approach to ensure comprehensive analysis and effective implementation of the ChaCha20 algorithm in Deep and Dark Web environments. The following framework will guide the research process: [3]

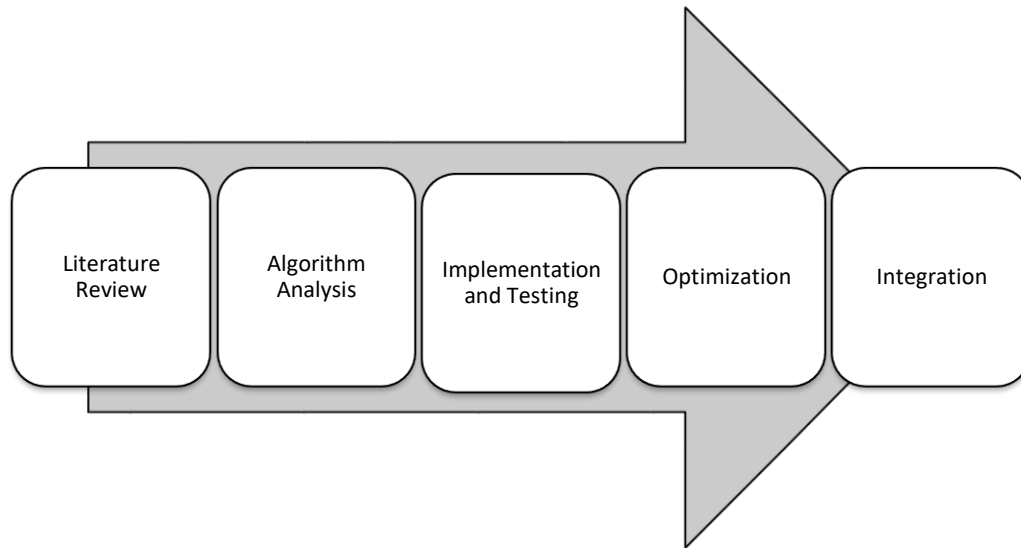


Figure 1.1: The Framework

1. Literature Review

A thorough review of existing research papers, official documentation, and case studies on ChaCha20 and its application in encryption protocols will be conducted. Additionally, research will focus on the security challenges within the Deep Web and Dark Web, highlighting current encryption techniques and their limitations. This phase will establish the foundational knowledge necessary for subsequent stages.

2. Algorithm Analysis

This phase will involve an in-depth examination of the internal structure of ChaCha20. Key aspects such as the use of XOR operations, bitwise rotations, and modular additions will be analyzed to understand how the algorithm achieves security and efficiency. The objective is to identify strengths and potential vulnerabilities in ChaCha20's design.

3. Implementation and Testing

The ChaCha20 algorithm will be implemented in simulated Deep Web and Dark Web environments using tools like TOR and I2P. Performance and security

tests will be conducted to evaluate how well ChaCha20 adapts to the network conditions typical of hidden services, measuring aspects like encryption speed, latency, and resource consumption.

4. Optimization

Techniques will be developed to improve ChaCha20's performance, particularly in resource-limited environments. This may involve leveraging hardware-specific instructions, such as SIMD (Single Instruction, Multiple Data), to accelerate encryption processes. The goal is to minimize computational overhead without compromising security.

5. Integration

The final phase focuses on incorporating ChaCha20 into existing security protocols that operate within the Deep Web and Dark Web, such as the TOR protocol. The algorithm's performance and effectiveness will be evaluated to ensure seamless integration and enhanced security across communication channels.

1.6 Project Motivation

The rapid expansion of encrypted communication and the increasing reliance on privacy-driven networks have underscored the need for stronger and more efficient encryption algorithms. In environments like the Deep Web and Dark Web, where anonymity and security are paramount, the choice of encryption directly impacts the safety and reliability of data transmission. However, many existing encryption algorithms face performance issues in resource-constrained environments or are vulnerable to emerging cryptographic attacks. [3]

ChaCha20 has gained recognition for its speed, simplicity, and high level of security, making it a compelling alternative to traditional ciphers like AES. Its ability to perform well on devices with limited computing power positions it as an ideal candidate for enhancing security protocols in the hidden layers of the internet. This project is motivated by the need to explore how ChaCha20 can be optimized and

integrated into the Deep and Dark Web, providing robust encryption that can withstand evolving threats while maintaining efficiency in decentralized and low-resource networks.

1.7 Tools and Technologies Used

- a. Programming Languages: Python, C/C++ (for algorithm implementation and performance testing)
- b. Simulation Environments: TOR Network, I2P (Invisible Internet Project)
- c. Cryptography Libraries: OpenSSL, Libsodium, and PyCryptodome (for ChaCha20 implementation)
- d. Performance Testing Tools: Wireshark (network analysis), Hyperfine (benchmarking tool)
- e. Development Environment: Visual Studio Code, Linux (Kali/Ubuntu)
- f. Hardware: ARM-based devices, Virtual Machines, and Raspberry Pi (for testing ChaCha20 in low-resource environments)

1.8 Scope of Research

Defining the scope of the project helps focus efforts on the core objectives without diverging into areas that may distract from the main goals. This project will focus on the following: **[3]**

1. Analyzing the ChaCha20 Algorithm:

The project will focus solely on the ChaCha20 algorithm without delving into other encryption algorithms such as AES or RSA. The focus will be on understanding how ChaCha20 works, its key generation methods, and its encryption/decryption processes. The goal is to gain a deeper understanding of its performance in Deep Web and Dark Web environments.

2. Performance Testing in Specific Environments:

The project will limit its performance testing to Deep Web and Dark Web environments, specifically targeting low-resource networks. The ChaCha20 algorithm will be tested in environments where network conditions are limited, such as low bandwidth or devices with constrained computational power, to assess its efficiency in these scenarios.

3. Security Analysis Against Known Cryptographic Attacks:

The project will analyze ChaCha20's security against known cryptographic attacks such as brute-force and side-channel attacks. It will not focus on developing or testing new attack methodologies against the algorithm.

4. Protocol Integration Within Commonly Used Security Protocols:

The project will focus on integrating ChaCha20 within established security protocols used in the Deep Web and Dark Web, such as the TOR protocol. The focus will be on how well ChaCha20 can enhance security within these protocols. [3]

1.9 Project Plan

The Project Plan for chacha20 algorithm using Gantt Chart is the following:

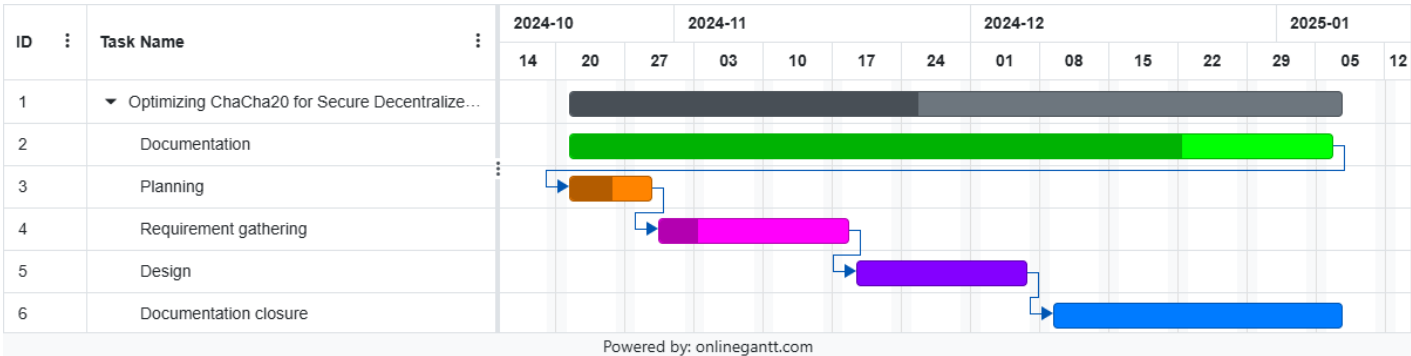


Figure 1.2 : Gantt chart

1.10 Project outline

The project begins with Chapter 1: Introduction, which provides a comprehensive overview of the project, including the problem statement, objectives, and the research framework. This chapter sets the foundation for the study by introducing the key areas of focus and the approach to addressing the identified challenges. Chapter 2: Literature Review follows, where previous studies on ChaCha20 and its applications in Deep Web and Dark Web networks are reviewed. This chapter highlights the existing research, identifies gaps, and provides context for the project's contribution. Finally, Chapter 3: Methodology describes the methods used for analyzing, implementing, and testing the ChaCha20 algorithm. It explains the process of evaluating the algorithm's performance and security within the specific environments of the Deep Web and Dark Web, detailing the steps taken for implementation and the testing protocols used to ensure reliable results.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview

cybersecurity is rapidly evolving, necessitating robust and efficient cryptographic solutions. Among the various algorithms, ChaCha20 has emerged as a leading choice due to its fast and secure encryption capabilities.

ChaCha20 is a stream cipher designed by Daniel J. Bernstein. Stream ciphers, unlike block ciphers like AES, operate on data byte by byte, generating a keystream that is XORed with the plaintext to produce the ciphertext.

ChaCha20 stands out for its high performance, strong security, and resistance to timing attacks, making it well-suited for various applications, including secure communication, data storage, and network encryption.

2.2 Critical evaluation (gap analysis)

A detailed analysis of ChaCha20 highlights its strengths in security, performance, and **usability** across multiple contexts. [4]

- **SECURITY:** ChaCha20 is designed to provide security equivalent to AES-256 (which uses a 256-bit key) but with better performance in software. It has been cryptanalyzed extensively and is considered highly secure, particularly against known attacks.
- **PERFORMANCE:** One of the key advantages of ChaCha20 over other ciphers like AES is its high performance on software. AES requires specific hardware acceleration (like AES-NI) to achieve optimal speeds, while ChaCha20 runs efficiently even without specialized hardware. This makes ChaCha20 particularly appealing for environments where hardware acceleration isn't available, such as on mobile devices or in web browsers.

- **USABILITY:** ChaCha20 has been adopted in various practical applications, including:
 - A. **TLS/SSL Protocols:** ChaCha20, combined with the Poly1305 authenticator, forms the ChaCha20-Poly1305 cipher suite, which is utilized in Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS) protocols. This combination offers authenticated encryption with associated data (AEAD), ensuring both confidentiality and integrity of communications. Notably, ChaCha20-Poly1305 is particularly advantageous on devices lacking hardware acceleration for AES, such as many mobile devices, providing enhanced performance and security.
 - B. **Virtual Private Networks (VPNs):** VPNs utilize encryption to secure data traffic between users and the internet. ChaCha20's performance benefits are particularly evident in VPN implementations like WireGuard, which employs ChaCha20 for its symmetric encryption needs. WireGuard's adoption of ChaCha20 contributes to its reputation for speed and efficiency, even on devices with limited processing power.
 - C. **Disk Encryption:** While ChaCha20 is primarily known for securing data in transit, its application in disk encryption is also noteworthy. By encrypting data stored on physical media, ChaCha20 helps protect sensitive information from unauthorized access. Its high-speed encryption capabilities ensure that data protection does not come at the cost of system performance.

2.3 Related Work

The development of custom hardware accelerators for cryptographic algorithms, such as AES (Advanced Encryption Standard) and ChaCha20, is a popular topic in engineering research. Many studies target FPGA (Field-Programmable Gate Array)-based platforms, hybrid SoCs (System-on-Chips), or ASICs (Application-Specific Integrated Circuits). For AES, Cowart et al [3], tested accelerators on a Zedboard (a

Zynq-7000-based development board) using two IP cores: a non-pipelined core for ECB and CBC modes and a pipelined core for CTR mode. They found the pipelined core to be nearly 7 times faster than the OpenSSL library on ARM, with a throughput of 350 MiB/s compared to 25 MiB/s for the non-pipelined core. Baskaran and Rajalakshmi fabricated an AES accelerator using 0.18- μ m CMOS technology, achieving 480 MiB/s at 330 MHz, supporting multiple modes like ECB, OFB, and CBC. ChaCha20 accelerators are less common but notable [3].

Silex Insight developed a ChaCha20-Poly1305 IP-core [7], supporting AEAD (Authenticated Encryption with Associated Data), but its exact throughput is unspecified [4]. Kanda and Ryoo designed a ChaCha20 accelerator for the Virtex 7 FPGA, achieving 515 MiB/s at 161 MHz with moderate resource usage (1692 LUTs and 566 registers)[5]. Another project proposed a co-processor for ChaCha20, BLAKE, and Skein, focusing on compactness and an alternative computation approach. These works highlight the advancements and challenges in creating efficient cryptographic hardware [6].

Source	Algorithm	Platform	Frequency	Performance
Kanda and Ryoo [7]	ChaCha20	Virtex 7	161 MHz	550 MiB/s
At et al. [6]	ChaCha20	Virtex 6	266 MHz	362 MiB/s
Strombergson [9]	ChaCha20	Cyclone V	60 MHz	159 MiB/s ²
This thesis [6]	ChaCha20	Cyclone V	50 MHz	120.5 MiB/s ³
Cowart et al. [3]	AES CBC	Zynq-7000	100 MHz	25MiB/s
Cowart et al. [3]	AES CTR1	Zynq-7000	100 MHz	350 MiB/s
Baskaran and Rajalakshmi [4]	AES	0.18- μ m CMOS	330 MHz	480 MiB/s

Table2.1: Summary of the performance characteristics for various hardware cyphers

2.3.1 Discussions of what has been done along the line of the problem

Significant work has been devoted to the development of cryptographic algorithms aimed at ensuring both security and performance. Algorithms like AES (Advanced Encryption Standard) have been widely adopted due to their robustness and versatility in various applications. However, AES's reliance on hardware acceleration for optimal performance has limited its applicability in resource-constrained environments, such as mobile devices or embedded systems [6].

ChaCha20 emerged as a solution to these limitations by providing an encryption mechanism that is both lightweight and efficient without compromising security. Research studies have extensively documented its superiority in software-only settings, highlighting its resistance to timing attacks and its ability to perform reliably across diverse platforms.

Several studies have explored the possibility of reducing the number of rounds in the ChaCha20 encryption algorithm to improve its efficiency while maintaining a reasonable level of security. ChaCha20, in its standard form, uses 20 rounds to ensure robust encryption. However, researchers have analyzed the security implications of using fewer rounds, such as 12 or 8, in specific scenarios. These reduced-round versions, often referred to as ChaCha12 and ChaCha8, are faster and require less computational power, making them suitable for resource-constrained environments like mobile devices or embedded systems. Despite their improved performance, these variants have been subjected to rigorous cryptanalysis to evaluate their resistance against various attacks. While ChaCha12 has been shown to retain strong security properties in most practical use cases, ChaCha8 is generally recommended only in situations where performance is prioritized over maximal security. This line of research highlights the trade-offs between security and efficiency in cryptographic algorithm design [6].

2.3.2 Issues and challenges faced by the other project

Despite the strengths of earlier cryptographic algorithms, several challenges have been identified [7]:

Title	Description
Performance Limitations in Software-Only Environments	AES often requires hardware-based acceleration to achieve satisfactory speeds, posing a challenge for systems without such capabilities.
Timing Attacks	Many older algorithms exhibit vulnerabilities to timing attacks, especially in environments with high computational variability.
Energy Efficiency	Resource-intensive algorithms can significantly drain power, making them unsuitable for battery-operated devices.
Implementation Complexity	The integration of some algorithms into lightweight systems often necessitates additional computational overhead, impacting overall efficiency.
Scalability	Earlier algorithms often struggled to adapt to the needs of emerging technologies, such as lightweight systems, constrained networks, and real-time encryption demands.

Table 2.2: Issues and challenges faced by the other project

2.3.3 Critical evaluation (gap analysis)

This table compares AES and ChaCha20, with ChaCha20 offering better energy efficiency, timing attack resistance, and ease of implementation, while AES performs well with hardware acceleration [8].

Aspect	AES	ChaCha20
Performance Improvements	Faster with hardware acceleration	Efficient in both hardware and software
Resistance to Timing Attacks	Moderate resistance	High resistance
Energy Efficiency	Higher power consumption	More energy-efficient
Ease of Implementation	Complex in lightweight systems	Easier to implement
Wide Applicability	Widely used in various systems	Versatile and suitable for modern use

Table 2.3: Compares AES and ChaCha20

2.4 Summary

The reviewed literature and evaluations demonstrate that ChaCha20 is a significant advancement in cryptographic algorithms. By addressing the limitations of prior solutions, particularly in resource-constrained environments, it has established itself as a versatile and reliable alternative to traditional block ciphers. Its high performance, robust security, and adaptability make it a preferred choice for modern cryptographic applications, particularly in environments where hardware acceleration is not feasible.

CHAPTER 3

METHODOLOGY

3.1 System Analysis

ChaCha20 is a classic stream cypher: it produces a stream of pseudo-random bytes (one-time pad or OTP) that is XORed with the plaintext to produce the cyphertext. Demonstrates how a 512-bit chunk of such a stream is generated according to RFC8439. First, the initial 512-bit state of ChaCha20 must be formed. This state consists of a few fields that are explained in Table[3.1]. Second, the initial state is transformed by 20 round functions of two types: even and odd. Each round function converts its 512-bit input into a 512-bit output. Finally, the initial state is added to the result of the last round function. To do the summation stage, both [15]

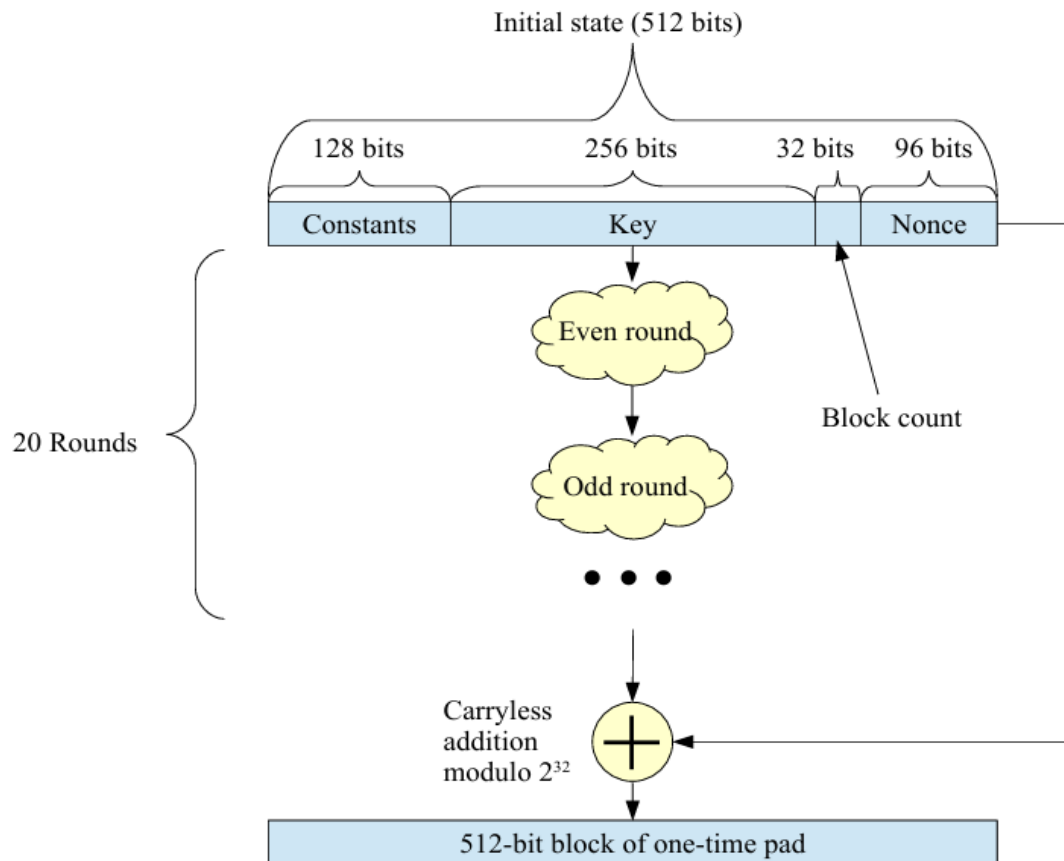


Figure 3.1: Graphical illustration of ChaCha20 algorithm

operands are viewed as arrays of 32-bit integer numbers with 16 elements.

The summation modulo 2^{32} is done element-wise between the arrays. The result of the summation is used to do encryption as follows:

$$C_i = P_i \oplus PAD_i$$

where C_i is the i -th bit of the ciphertext; P_i is the i -th bit of the plaintext; and PAD_i is the i -th bit of ChaCha20 result. [13]

Name	Bit size	Byte size	Description
Constant	128 bits	16 bytes	A constant part of the state. It is always initialized with the ASCII values of the characters of the following string: ‘expand 32-byte k’
Key	256 bits	32 bytes	The secret key.
Block count	32 bits	4 bytes	A counter that starts from 0 and is incremented to generate the next chunks of OTP. There are 2^{32} unique values of this field, so the maximum length of the entire OTP with the same secret key and nonce is 256 GiB.
Nonce	96 bits	12 bytes	A unique number that can be changed to generate a new OTP with the same key. Using each value of nonce no more than once is crucial for providing a high level of security.

Table 3.1: Fields of ChaCha20 initial state

To generate the next 512-bits of OTP, the block count field of the initial state is incremented and the process repeats. The final result of ChaCha20 is extremely sensitive to changes of the initial state: flipping even a single bit of the input leads to an unpredictable change of the result. Moreover, despite the round functions being reversible, it is impossible to convert the result of ChaCha20 back to the initial state, because of the summation stage. These two properties make ChaCha20 suitable for stream encryption.

The round functions consist of one layer of quarter rounds, each working on 1/4 of the rounds input. **Figure(3.2)** shows the structure of the quarter round. The input of the quarter round consists of 4 32-bit words taken from the input of the round. Even and odd rounds supply the input to the quarter rounds differently. [9]

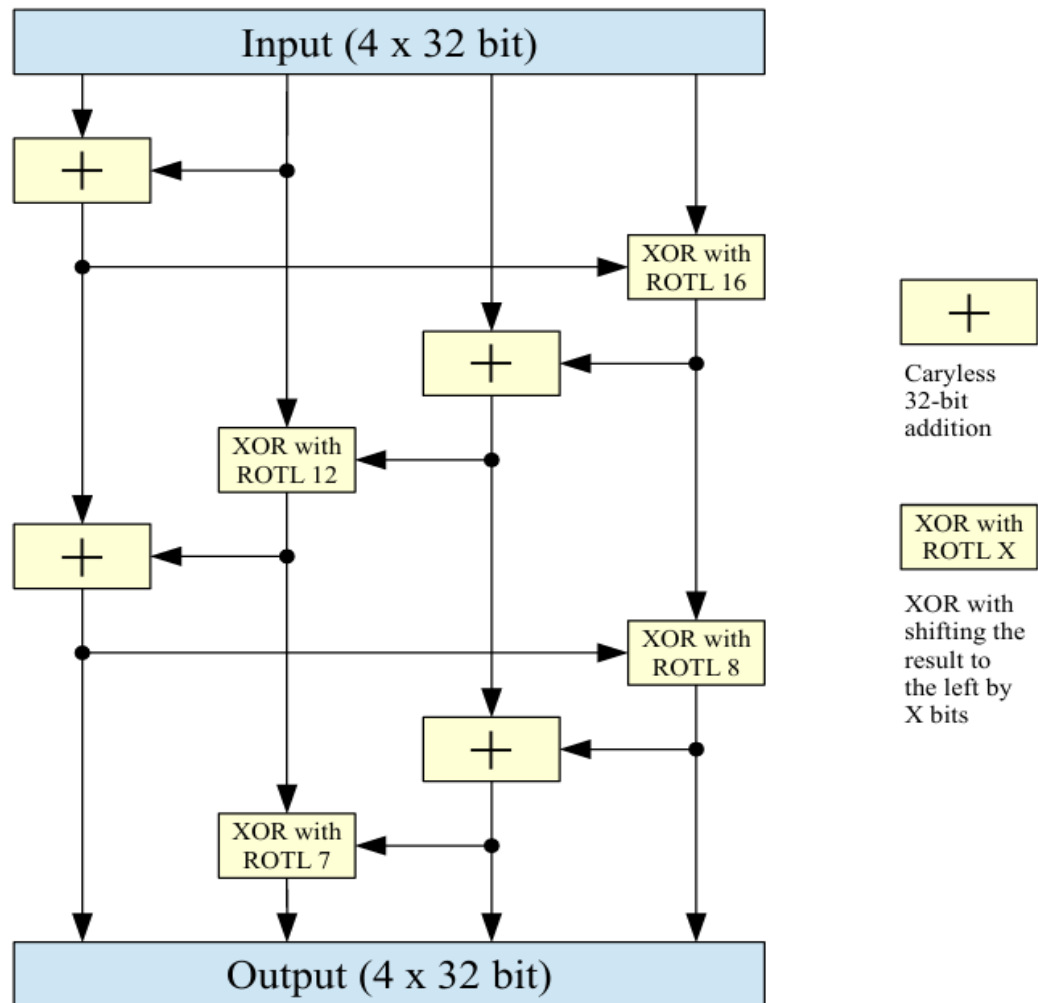


Figure 3.2: Graphical illustration of ChaCha20 round

show how the input words of the even and odd rounds are supplied to their quarter rounds.

High performance of ChaCha20 can be ascribed to the fact that it is based on a few primitive operations that execute in a minimum number of clock cycles on modern 32-bit and 64-bit processors:

- Carryless addition of two 32-bit numbers: $a + b \bmod 2^{32}$
- Exclusive OR between two 32-bit numbers: $a \oplus b$.
- Rotation of a 32-bit number by a fixed number of positions: $\text{ROTL}^n(a)$.

Quarter round number	Input words for the quarter round
1	0, 5, 10, 15
2	1, 6, 11, 12
3	2, 7, 8, 13
4	3, 4, 9, 14

Table3.2: ChaCha20 even round input words

Quarter round number	Input words for the quarter round
1	0, 4, 8, 12
2	1, 5, 9, 13
3	2, 6, 10, 14
4	3, 7, 11, 15

Table3.3: ChaCha20 odd round input words

Even though having only such simple operations in an encryption algorithm may seem insecure, Bernstein points out that his operators can simulate any circuit and therefore provide the same security level as a set of more complex operations.[9]

3.1.1 Requirement elicitation techniques

- Stakeholder Identification [10]

- a) **First Users:** Cryptographers and Researchers, Software Developers, Hardware Designers, Organizations and Standards Bodies.
- b) **Second Users:** Security Professionals, End Users, Governments and Regulators, Open Source Communities.

-Techniques Used :

- a) **Interviews:** Conducted structured interviews with Software Developers and Cryptographers and Researchers to understand their needs and pain points regarding website security.
- b) **Surveys:** Distributed surveys together broad input from various stakeholder.
- c) **Document Analysis:** Reviewed existing security protocols, compliance requirements, and industry standards.

- d) **Observation:** Shadowed security analysts during vulnerability assessment tasks to understand their workflow and challenges.
- e) **Workshops:** Organized workshops with stakeholders to brainstorm features and requirements.

3.1.2 Specific Requirements

The requirements for implementing and using Chacha20 are as follows:

3.1.2.1 External Interfaces

- **User Interface (UI)**
 - **Purpose:** The user interface allows users to configure, monitor, and interact with the ChaCha20 encryption system
 - **Inputs:**
Plaintext data to be encrypted (either files or real-time data).
Key and nonce selection.
Option to select encryption rounds.
 - **Outputs:**
Encrypted ciphertext displayed or saved.
Status messages (e.g., success, error, encryption time).
 - **Technology:**
This can be implemented as a command-line interface (CLI), a graphical user interface (GUI), or even a web-based interface, depending on the use case.[10]
- **Network Interface**
 - **Purpose:** The network interface defines how the ChaCha20 encryption algorithm communicates with remote devices, decentralized applications, or networks, such as web servers or IoT devices. This is especially relevant for systems utilizing encrypted communication (e.g., VPNs or TLS protocols).[10]

- **Protocol Usage:**
 - TLS/SSL Protocols: ChaCha20, as part of the ChaCha20-Poly1305 cipher suite, is often used in secure communication protocols like TLS/SSL. The system can interface with a network by handling encrypted data sent over HTTP(S) or other secure protocols.
 - VPN Protocols: If implemented in a VPN solution, ChaCha20 can be used to secure communications between clients and servers by encrypting packets.
 - **Inputs/Outputs:**
 - Encrypted data sent/received over the network.
 - Handshake and key-exchange information to securely generate keys and nonces for encryption.
 - **Technology:** The system may interact with a network stack or utilize existing libraries such as OpenSSL or custom implementations of the TLS/SSL or VPN protocols.[11]
- **Hardware Interface**
 - **Purpose:** The hardware interface outlines how ChaCha20 interacts with physical devices or custom hardware accelerators (e.g., FPGAs, ASICs) to perform encryption tasks efficiently.
 - **Components:**
 - Hardware Accelerators: ChaCha20 might be implemented in hardware to accelerate the encryption process. This is typically done on platforms such as FPGAs or ASICs.
 - FPGA Interface: If implemented on an FPGA, the system will interface with a hardware description language (HDL) module that performs the cryptographic operations. The system might communicate with the FPGA through standard I/O protocols like SPI or PCIe.
 - ASIC Interface: For a dedicated ASIC solution, the interface may involve communication through GPIO or high-speed serial connections.[11]
 - **Inputs/Outputs:**
 - The system may receive plaintext data, encryption keys, and nonces to be encrypted from a software application or external device.

Encrypted data (ciphertext) can be returned to the software system or external device for transmission or storage.

- **Technology:** The system will need hardware abstraction layers (HAL) or driver software to communicate with hardware devices. [11]

- **Software Interface**

- **Purpose:** ChaCha20 may be integrated into larger software systems such as operating systems, application software, or cryptographic libraries. The software interface outlines how ChaCha20 interacts with these systems to perform encryption tasks.

Libraries and APIs:

ChaCha20 can be exposed as a cryptographic function or API (e.g., via a C or Python library) that allows other software applications to access its functionality.

It could be integrated into a larger security library like OpenSSL, BouncyCastle, or Libsodium.

- **Inputs/Outputs:**
Inputs include data to be encrypted, encryption keys, and nonces.
Outputs include ciphertext and possibly authentication tags (if combined with an AEAD mode like ChaCha20-Poly1305).
- **Technology:** ChaCha20 could be implemented as a module in a cryptographic library with predefined functions for encryption and decryption. [11]

User Interface (UI)	Provides users with an easy way to interact with the encryption process (e.g., inputting data, selecting keys)
Network Interface	Facilitates secure communications via protocols like TLS/SSL or VPNs, where ChaCha20 is used to encrypt data
Hardware Interface	Enables ChaCha20 to be accelerated via hardware platforms (e.g., FPGA, ASIC), improving performance
Software Interface	Allows integration of ChaCha20 into other software systems through APIs or cryptographic libraries

Table 3.4: Types of Interfaces and Their Roles in ChaCha20 Encryption

3.1.2.2 Functional Requirements

Functional requirements define the essential tasks and functionalities that the ChaCha20 encryption system must fulfill to meet user and system expectations. These requirements focus on the system's operations, ensuring that it performs its cryptographic tasks effectively and securely.[12]

- **Improved Encryption Performance**

The system must provide fast encryption and decryption capabilities. ChaCha20 should operate efficiently even in software environments without hardware acceleration, offering superior performance compared to other encryption algorithms like AES in such contexts.

- Requirements**

The system must be able to process data at a high throughput (e.g., several hundred MiB/s on common processor architectures) to ensure low latency encryption for various applications such as secure communications, VPNs, and data storage.

The system must utilize minimal CPU resources to avoid excessive load on devices, especially in constrained environments like mobile or embedded systems. [12]

- **Provide Security for Data Sent Over Decentralized Networks**

The system must guarantee the confidentiality and integrity of data exchanged over decentralized or distributed networks (e.g., P2P networks, decentralized applications). [12]

- Requirement:**

The ChaCha20 encryption must be resistant to various types of attacks (e.g., brute force, side-channel attacks, and chosen plaintext attacks).

The system must ensure that data in transit is encrypted in such a way that it cannot be easily intercepted or tampered with.

ChaCha20 should be capable of working in conjunction with additional protocols (e.g., TLS/SSL, WireGuard for VPNs) to secure communications over decentralized networks. [12]

- **Efficient Use of System Resources**

The system must be optimized for resource-constrained environments, especially where processing power and memory are limited (e.g., mobile devices, IoT devices). [12]

- Requirement:**

- The ChaCha20 algorithm must have a low memory footprint and minimal power consumption, making it suitable for battery-operated devices or low-powered systems.

- The system must efficiently manage CPU usage, avoiding high overhead during encryption and decryption operations. [12]

3.1.2.3 Design Constraints

Design constraints define the limitations and restrictions that must be considered during the development of the ChaCha20 encryption system. These constraints impact how the system is implemented, how it interacts with hardware and software, and how it is integrated into existing infrastructures. [13]

- **Hardware Limitations**

The system must be designed to function effectively on a variety of hardware platforms, ranging from high-performance servers to low-powered mobile devices, embedded systems, and IoT devices. The hardware limitations can significantly influence the performance and efficiency of the ChaCha20 algorithm. [13]

- **Constraints:**

- 1. **Processing Power**

- The system should be able to perform ChaCha20 encryption on devices with limited processing capabilities, ensuring that it runs efficiently even on older or resource-constrained hardware. [13]

- 2. **Memory Usage**

- The system must operate within the memory constraints of the target device. Memory usage must be kept minimal to avoid performance degradation or crashes on devices with limited RAM.

3. Code Efficiency

Given that ChaCha20 is commonly used in environments where high performance and low resource consumption are critical (such as mobile devices or embedded systems), the code must be optimized to run efficiently.

Constraints:

Code Size: The implementation should be compact, with minimal overhead, ensuring that the code does not consume too much memory or processing time.

Execution Speed: The algorithm should be highly optimized to minimize computational cycles. For example, operations like the 32-bit rotations and XOR operations should be as fast as possible.

- **Constraints:**

1. Code Size

The implementation should be compact, with minimal overhead, ensuring that the code does not consume too much memory or processing time. [14]

2. Execution Speed

The algorithm should be highly optimized to minimize computational cycles. For example, operations like the 32-bit rotations and XOR operations should be as fast as possible.

• **Security Requirement**

While ChaCha20 is already secure, the implementation must respect specific security requirements to avoid vulnerabilities that could be introduced during integration or usage.

- **Constraints:**

- **Resistance to Side-Channel Attacks:** The implementation must be designed to resist side-channel attacks (e.g., timing attacks, power analysis). The algorithm must be implemented in such a way that no information is leaked about the encryption key through such channels.
- **Nonce Management:** Nonce generation and management must be done securely to prevent nonce reuse or collisions, which could lead to vulnerabilities.

- **Secure Key Storage:** The system must securely store and manage cryptographic keys. Keys should not be exposed or easily recoverable in any implementation or communication process. [14]

3.1.3 Non-functional Requirement

Non-functional requirements refer to the quality attributes that define how a system performs its functions, rather than specifying what functions it performs. These requirements are essential for ensuring the system's overall effectiveness, especially in contexts such as security and performance, and are critical when enhancing the ChaCha20 algorithm for decentralized networks.

-Key Non-functional Requirements:

1. Performance:

Definition: Performance refers to the system's speed, processing efficiency, and response time. This requirement indicates how efficiently the system executes its functions within a specific time frame. [15]

Importance: For a decentralized system using the ChaCha20 algorithm, performance is crucial as encryption and decryption operations must be fast without compromising system responsiveness.

Example: The system should encrypt data in less than a second for large datasets to minimize latency during communication processes. [15]

2. Security:

Definition: Security involves protecting sensitive data from unauthorized access, ensuring data integrity, and resisting attacks such as those targeting decentralized networks.

Importance: Since the ChaCha20 algorithm is used to protect data across the network, it is crucial to provide strong encryption that resists various types of attacks.

Example: The system should implement ChaCha20 encryption to ensure that data remains secure and inaccessible to unauthorized users, even in a decentralized environment. [15]

3. Scalability:

Definition: Scalability refers to the system's ability to handle increased user or data load without degrading performance.

Importance: As decentralized networks grow and more nodes join, the system must continue to perform efficiently without significantly affecting the speed of encryption or security.

Example: The system should be able to handle increased network traffic or a larger number of users without significant performance degradation. [15]

4. Stability:

Definition: Stability ensures that the system remains reliable and continues to perform as expected even in complex or fluctuating network conditions.

Importance: Decentralized networks often face unpredictable changes, and the system must be able to remain stable and function consistently across different environments.

Example: The system must maintain stable encryption and decryption processes even when some nodes in the network fail or experience delays. [15]

5. Flexibility:

Definition: Flexibility refers to the system's ability to adapt to future changes in network protocols or technologies.

Importance: The system should be adaptable, allowing for improvements or modifications in encryption methods or network protocols without requiring significant overhauls

Example: The system should support future versions of ChaCha20 or other encryption algorithms without requiring major changes to the core architecture. [15]

3.1.3.1 Data flow Diagrams(DFDs)

-DFD Level 0 (High-Level View):

Description: At Level 0, the system is represented as a single process where data flows in and out without detailed sub-processes. [16]

External Entities:

User/Application: Provides the plaintext, key, and nonce.

Encrypted Data Recipient: Receives the encrypted data.

Process:

ChaCha20 Encryption: Encrypts the data using the key and nonce.

Data Flows:

Plaintext, Key, Nonce (from User/Application)

Encrypted Data (to Encrypted Data Recipient) [16]

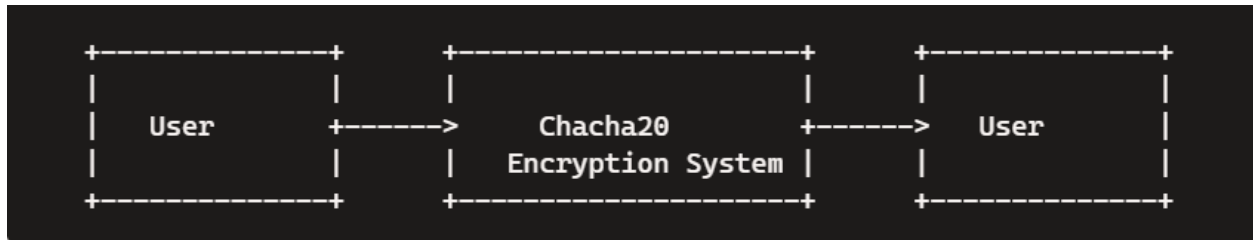


Figure 3.3: DFD Level 0

-DFD Level 1 (Detailed View):

Description: In Level 1, the main encryption process is broken down into sub-processes such as Key Setup, Key Stream Generation, and Encryption.

Processes:

Key Setup: Initializes the key and nonce.

Key Stream Generation: Generates the key stream from the key and nonce.

Encryption: Applies the key stream to the plaintext.

Output: Sends the encrypted data to the recipient.

Data Flows:

Key, Nonce, Plaintext (from User/Application)

Key Stream (between Key Setup and Key Stream Generation)

Encrypted Data (to Encrypted Data Recipient. [16]

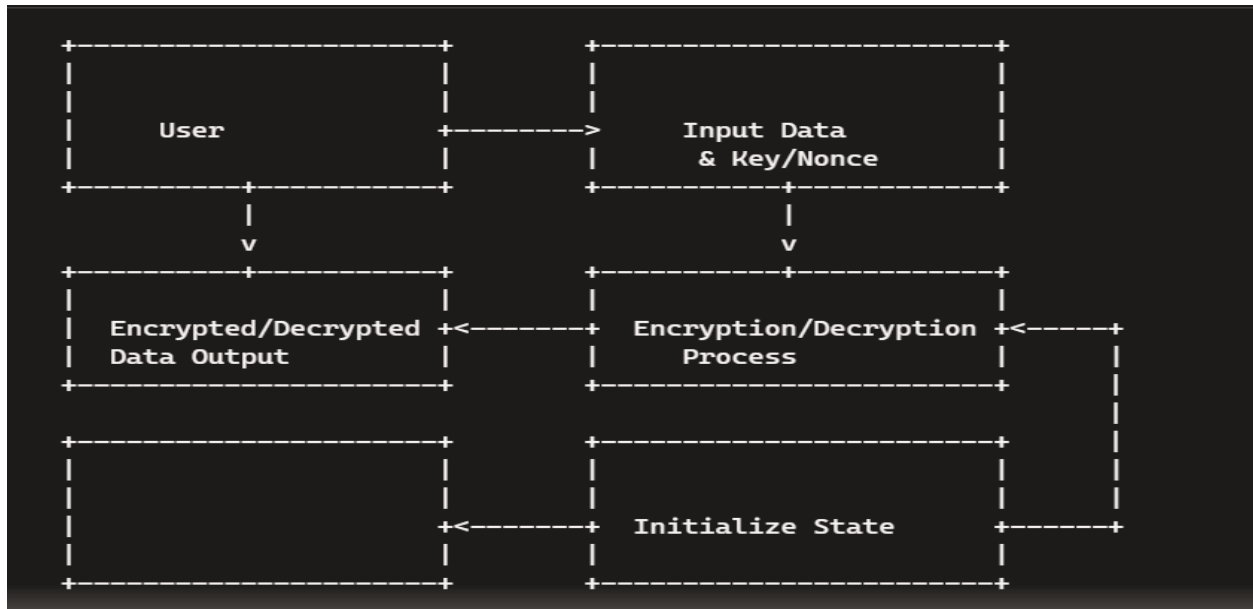


Figure 3.4: DFD Level 1

-DFD Level 2 (Detailed Sub-Processes):

Description: Level 2 further decomposes the sub-processes from Level 1 into finer details.

Processes:

Key Setup: Includes Key & Nonce Preparation and Key Scheduling.

Key Stream Generation: Includes Block Generation, Round Operations, and XOR with Plaintext.

Encryption: Apply Key Stream to Plaintext.

Data Flows:

Key, Nonce, Plaintext (from User/Application)

Key Stream (generated in Key Stream Generation)

Encrypted Data (to Encrypted Data Recipient) **[16]**

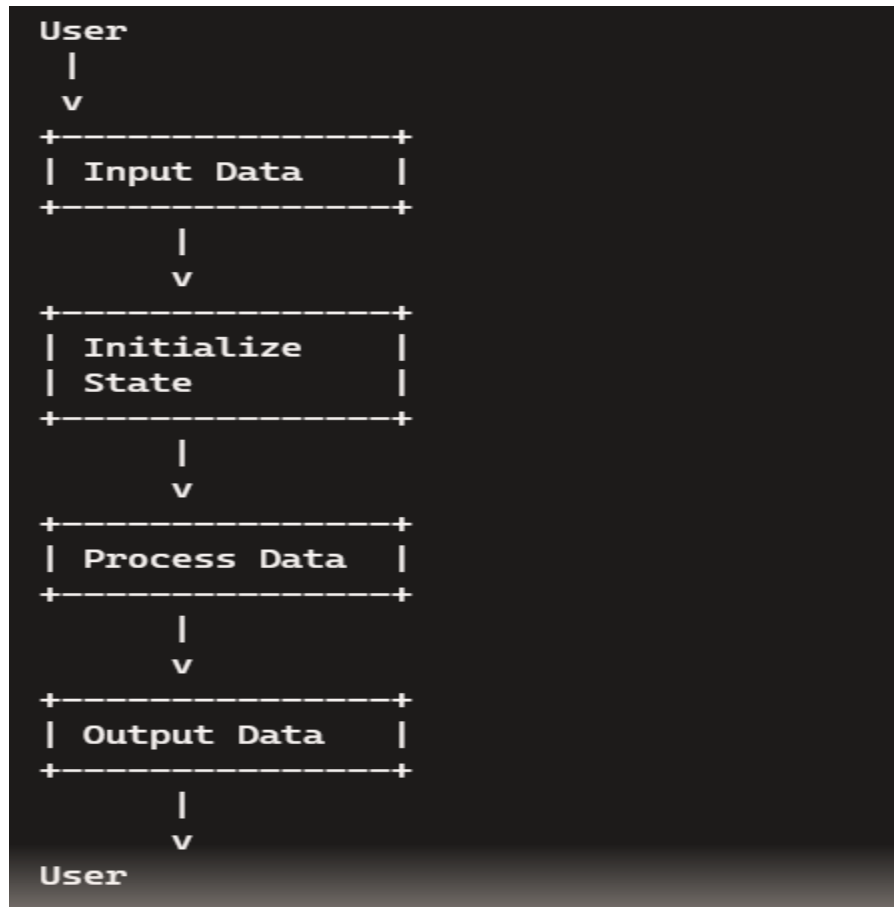


Figure 3.5: DFD Level 2

3.1.3.2 System Design

User Interface (UI): Allows users to input plaintext, keys, nonce, and receive the encrypted/decrypted output.

Backend Service: Manages the encryption/decryption process and state initialization.

Data Storage: Securely stores keys, nonces, and possibly encrypted data.[17]

-Architecture:

Frontend:

User Input Forms: To input plaintext, key, and nonce.

Display Output: Shows the encrypted or decrypted data.[17]

Backend:

API Endpoints: To handle requests for encryption and decryption.

Encryption Module: Implements Chacha20 algorithm.

State Initialization: Handles key, nonce, and counter setup.

Database:

Secure Storage: For keys and nonces, ensuring data is encrypted at rest. [17]

-Flow Diagram:

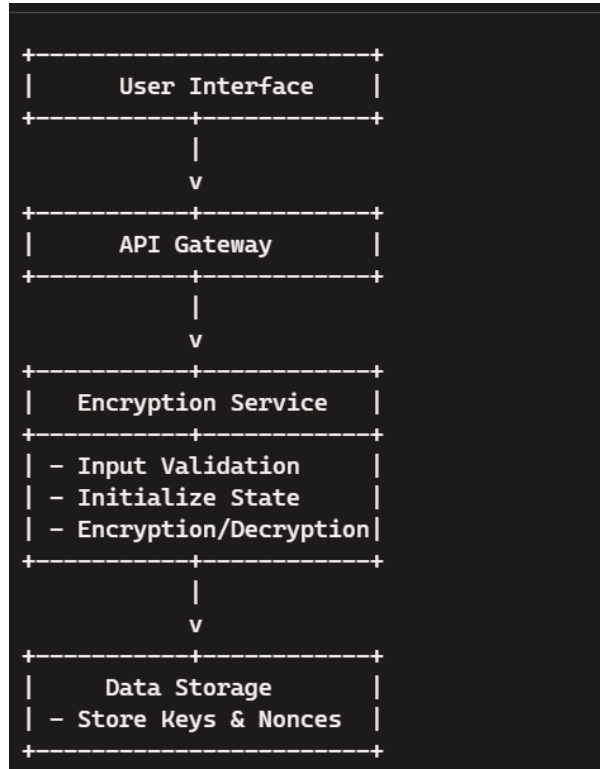


Figure 3.6: Flow Diagram

-Use Cases diagram:



Figure 3.7: Use Cases diagram

-Activity Diagram:

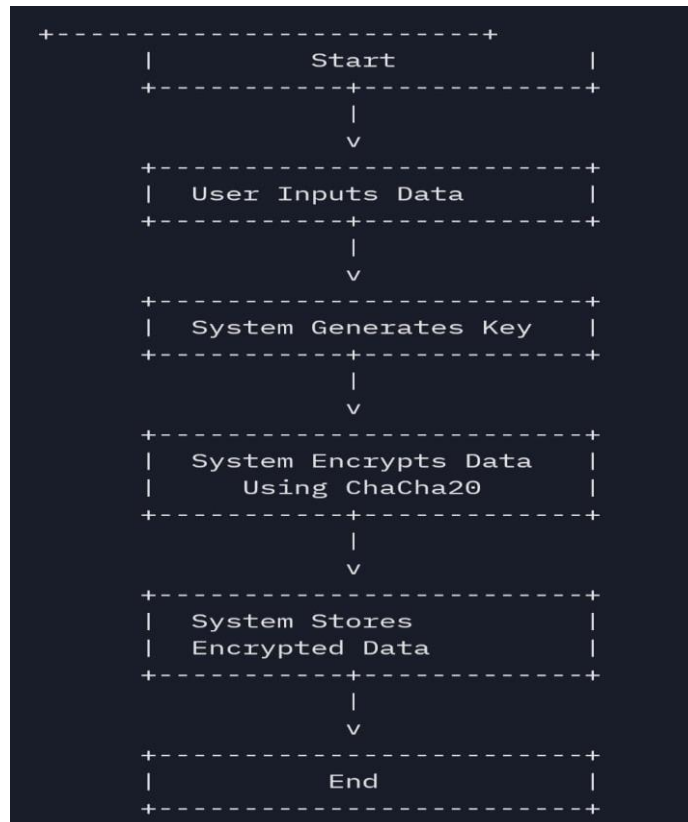


Figure 3.8: Activity Diagram

CHAPTER 4

SYSTEM IMPLEMENTATION

This chapter provides a comprehensive explanation of how the ChaCha20/12 encryption benchmarking tool was implemented. It details the programming environment, key libraries used, rationale behind design decisions, and the flow of the system from file selection to result display via a graphical interface.

4.1 Programming Language and Libraries

4.1.1 Justification for Using Python

Python was chosen as the primary language for this project for several important reasons: [18]

- **Simplicity and Clarity:** Its clean syntax supports rapid development and improves code readability.
- **Rich Standard and External Libraries:** Python offers powerful built-in and third-party libraries such as **tkinter** for GUI development, **time** for performance measurement, and **cryptography** for secure encryption.
- **Cross-Platform Support:** Python applications can run on multiple operating systems with minimal changes.
- **Strong Community and Documentation:** Ample support resources make development and debugging efficient.

4.1.2 Justification for Using the cryptography Library

The cryptography library was used to implement ChaCha20/12 encryption and decryption due to the following benefits: [18]

- **Modern and Secure:** It follows up-to-date cryptographic standards, ensuring secure handling of data.
- **Ease of Use:** Provides a high-level API to access complex algorithms like ChaCha20 without in-depth low-level programming.

- **Reliable Performance:** Efficient implementation with support for memoryview objects, minimizing memory overhead

4.1.3 Justification for Using tkinter

tkinter was used to design the graphical user interface for the benchmarking tool. It was selected because: [18]

- **Native Python Integration:** It comes bundled with Python, eliminating the need for external dependencies.
- **Ease of Development:** Simple to use for building functional desktop applications.
- **User-Friendly Experience:** Enables interaction via buttons, file selectors, and text display fields, making the tool accessible to non-technical users.

4.2 System Workflow and Components

The tool follows a structured process that allows users to select multiple text or JSON files, benchmark their encryption and decryption times using ChaCha20/12, and view the results instantly. [18]

4.2.1 File Benchmarking Logic

1. **File Selection:** The user selects .txt or .json files via a graphical file dialog.
2. **File Reading:** The content of each file is read in binary mode.
3. **ChaCha20/12 Setup:**
 - A 256-bit key and a 128-bit nonce are generated using `os.urandom()`.
 - The ChaCha20 algorithm is initialized using the cryptography library.
4. **Benchmarking Process:**
 - The encryption and decryption times are measured using `time.perf_counter()`.
 - A comparison is made to ensure the decrypted data matches the original content.
5. **Result Display:** The result for each file (encryption and decryption time) is displayed in a scrollable text area in the GUI.

4.2.2 Pseudocode Representation

```
BEGIN
  User selects .txt/.json files through GUI
  FOR each selected file:
    Read file as binary data
    Generate ChaCha20 key and nonce
    Start encryption timer
    Encrypt the data
    Stop timer and save encryption time
    Start decryption timer
    Decrypt the data
    Stop timer and save decryption time
    IF decrypted data matches original:
      Display success and timings
    ELSE:
      Display error message
END
```

Figure 4.1 : Pseudocode Representation

4.2.3 GUI Components Overview

- **Main Window:** Displays the application title and instructions.
- **File Selector Button:** Triggers file selection via system dialog.
- **Result Textbox:** A multi-line area to show per-file encryption and decryption times.
- **Layout Design:** Light color theme, clear fonts (Segoe UI, Courier New) for professional readability

CHAPTER 5

SYSTEM EVALUATING AND TESTING

This chapter discusses the testing and evaluation phase of the ChaCha20/12 file encryption benchmarking tool. The primary objective of testing was to ensure the system's correctness, stability, and reliability in both functional and performance aspects. Different test plans were implemented to verify that the system operates as intended across multiple scenarios.

5.1 Testing Plan

A structured testing plan was developed to evaluate the key functionalities of the system, particularly the accuracy of encryption/decryption operations and the responsiveness of the graphical interface. [19]

5.1.1 Functional Testing

This type of testing focused on ensuring that each feature of the system works correctly. Key components tested include: [19]

- **File Handling:** The system was tested with valid and invalid .txt and .json files to confirm its ability to read data and handle exceptions (e.g., unreadable or corrupted files).
- **Encryption/Decryption Logic:** Ensured that the ChaCha20/12 algorithm correctly encrypts and decrypts files, and that decrypted data exactly matches the original input.
- **Result Output:** Verified that performance results (encryption/decryption time) are accurately calculated and displayed in the GUI.

5.1.2 GUI Testing

The graphical user interface was tested to validate its usability and stability: [19]

- **Responsiveness:** Verified that the interface remains responsive during multiple file selections and processing tasks.
- **Error Handling:** Tested the system's ability to display clear messages when a file cannot be read or decrypted.
- **User Feedback:** Confirmed that users receive immediate feedback after each file is processed, via the result textbox.

5.1.3 Performance Testing

This part of testing focused on measuring how the system performs with varying file sizes and contents: [19]

- **Small File Testing:** Used files smaller than 1 MB to test low-latency behavior.
- **Medium File Testing:** Used files between 1 MB to 10 MB to measure consistent performance under typical conditions.
- **Large File Testing:** Tested files larger than 100 MB to evaluate processing speed and memory usage.

5.2.1 File-Based Testing Scenarios

Test Case	Input Type	Expected Outcome	Result
Valid .txt file	1 MB plain text	Successful encryption & decryption	Passed
Valid .json file	Structured JSON	Successful encryption & decryption	Passed
Empty file	0 bytes	Skipped with appropriate error message	Passed
Corrupted file	Invalid binary	Handled gracefully with error message	Passed
Multiple files at once	Mixed sizes	Each file processed individually with results	Passed

Table 5.1: File-Based Testing Scenarios

5.2.2 Decryption Validation

Each file processed is automatically decrypted after encryption. A byte-by-byte comparison is performed between the original and decrypted content to ensure correctness. Any mismatch triggers an error message and skips the faulty result. [20]

5.2.3 Performance Consistency

Benchmark tests showed that the encryption and decryption times increase proportionally with file size. This linear performance confirms the efficiency and scalability of the ChaCha20/12 algorithm for real-world file sizes. [20]

5.2.4 Performance Screenshots (Before and After Optimization)

To clearly show the impact of optimizing the ChaCha20/12 encryption algorithm, the following screenshots display encryption and decryption times recorded during the testing phase. These results were captured before and after performance improvements were made to the code.

In the first screenshot, we can see the original version of the algorithm running on several files. The encryption and decryption times are noticeably higher, especially for medium and large-sized files. This behavior was expected due to the lack of optimizations in how the algorithm handled file processing and memory usage.

```
Processing: C:/Users/LENOVO/Desktop/data1MB.txt
Data size: 1048576 bytes
Encryption time: 0.002394 s
Decryption time: 0.001874 s
Decryption successful for C:/Users/LENOVO/Desktop/data1MB.txt

Final Results:
Filenames: ['data1MB.txt']
Encryption times: [0.0023944000713527203]
Decryption times: [0.0018742000684142113]
PS C:\Users\LENOVO\Desktop>
```

Figure 5.1: shows the results before optimization.

In the second screenshot, taken after applying optimization techniques (such as better memory handling and refined loop structures), the encryption and decryption times are clearly reduced. The results show faster performance across all file sizes while maintaining correct functionality and accuracy.

```
C:\Users\Owner\anaconda3\python.exe "C:\U
File: data1MB.txt
Encryption time: 0.001732 seconds
Decryption time: 0.000495 seconds
```

Figure 5.2: presents the results after optimization.

These screenshots confirm that the applied optimizations successfully improved the efficiency of the encryption process without affecting the integrity of the results. The performance boost is particularly important for resource-constrained environments, where every millisecond matters.

CHAPTER 6

CONCLUSIONS

6.1 Conclusion

This project successfully developed and implemented a desktop-based benchmarking tool to evaluate the performance of the ChaCha20/12 encryption algorithm. The system enables users to select text and JSON files and measure encryption and decryption times using a simple and intuitive graphical interface built with tkinter. The encryption logic was based on the cryptography library, ensuring both accuracy and modern cryptographic standards.

Throughout the implementation, the system proved to be functional, responsive, and accurate. It validated the integrity of the encrypted data by automatically decrypting each file and comparing it against the original. The measured results confirmed the expected linear relationship between file size and processing time, showcasing the scalability of the ChaCha20/12 algorithm.

Furthermore, the use of memoryview and optimized design choices helped minimize memory overhead, making the tool efficient even when processing large files. The GUI offered a user-friendly experience that does not require technical expertise, making the tool accessible to a broad range of users.

6.2 Future Work

Although the current version of the system fulfills its intended purpose, several enhancements can be made in future versions to improve functionality, usability, and versatility:[21]

- **Support for Additional File Types:** Expand the tool to support other common file formats such as PDFs, images, and binary files to evaluate encryption performance in more diverse scenarios.
- **Multi-threading and Parallel Processing:** Introduce multi-threading to process multiple files simultaneously, significantly reducing processing time and enhancing user experience.
- **Detailed Performance Reports:** Allow users to export benchmarking results in formats like CSV or PDF for further analysis or documentation.
- **Algorithm Comparison Mode:** Integrate support for other encryption algorithms such as AES and Blowfish to allow side-by-side performance comparison with ChaCha20/12.
- **Command-Line Interface (CLI) Mode:** In addition to the GUI, a CLI version can be developed to provide flexibility for advanced users and developers who prefer script-based usage.
- **Cross-Platform Distribution:** Package the application for easy installation on different operating systems (e.g., Windows, macOS, Linux) using tools like PyInstaller or cx_Freeze.
- **User Settings Panel:** Add configurable options in the GUI such as algorithm parameters (key size, nonce size), output verbosity, or color themes for better customization.

- **Security Enhancements:** Add secure logging, hashed result verification, and optional file wiping for scenarios that require higher levels of data protection.

These improvements would not only broaden the tool's capabilities but also increase its adoption in academic, commercial, or cybersecurity-related use cases. With continued development, this benchmarking tool can become a powerful and flexible utility for evaluating lightweight encryption methods like ChaCha20/12 in real-world environments.

REFERENCES

- [1] <https://en.m.wikipedia.org/wiki/Salsa20>
- [2] <https://csrc.nist.gov/>
- [3] R. Cowart, D. Coe, J. Kulick, and A. Milenkovic, An Implementation and Experimental Evaluation of Hardware Accelerated Ciphers in All-Programmable SoCs, in Proceedings of the SouthEast Conference, ser. ACM SE 17, Kennesaw, GA, USA: Association for Computing Machinery, 2017, pp. 3441. doi: 10.1145/3077286.3077297. [Online]. Available: http://www.ece.uah.edu/~milenska/docs/rc_acmse17.pdf
- [4] S. Baskaran and P. Rajalakshmi, Hardware-Software Co-Design of AES on FPGA, in Proceedings of the International Conference on Advances in Computing, Communications and Informatics, ser. ICACCI 12, Chennai, India: Association for Computing Machinery, 2012, pp. 11181122, isbn: 9781450311960. doi: 10.1145/2345396.2345575
- [5] (Nov. 5, 2018). Silex Insight unveils high-throughput version of Chacha20 Poly1305 authenticated encryption, Silex Insight, [Online]. Available: <https://www.silexinsight.com/silex-insight-unveils-high-throughput-version-chacha20-poly1305-authenticated-encryption> (visited on 05/18/2020).
- [6] M. Bellare and C. Namprempre, Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm, Journal of Cryptology, vol. 21, pp. 469491, 4 2008. doi: 10.1007/s00145-008-9026-x. [Online]. Available: <https://cseweb.ucsd.edu/~mihir/papers/oem.pdf>.
- [7] G. Kanda and K. Ryoo, High-Throughput Low-Area Hardware Design of Authenticated Encryption with Associated Data Cryptosystem that Uses ChaCha20 and Poly1305, International Journal of Recent Technology and Engineering (IJRTE), vol. 8, pp. 8694, 2S6 2019, issn: 2277-3878. doi: 10.35940/ijrte.B1017.0782S619. [Online]. Available: <https://www.ijrte.org/wp-content/uploads/papers/v8i2S6/B10170782S619.pdf>
- [8] N. At, J. Beuchat, E. Okamoto, I. San, and T. Yamazaki, Compact Hardware Implementations of ChaCha, BLAKE, Three sh, and Skein on FPGA, IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 61, no. 2, pp. 485 498, 2014.

- [9] J. Strombergson. (2016). Verilog 2001 implementation of the ChaCha stream cipher, GitHub, [Online]. Available: <https://github.com/secworks/chacha> (visited on 05/15/2020).
- [10] <https://csrc.nist.gov/pubs/fips/197/final>
- [11] <chromeextension://efaidnbmnnnibpcajpcglclefindmkaj/https://theswissbay.ch/pdf/Books/Computer%20science/Cryptography/SeriousCryptography.pdf>
- [12] Title: "ChaCha20 and Poly1305", Author: Daniel J. Bernstein
URL: <https://cr.yp.to/chacha/chacha-20080128.pdf>
- [13] Source: The adoption of ChaCha20 in TLS 1.3 and its comparison to AES. Title: "ChaCha20 and Poly1305 for IETF protocols" Authors: Adam Langley, Justin A. P. Wright, and others. URL: <https://tools.ietf.org/html/rfc7539>
- [14] Source: ChaCha20's use in secure decentralized systems such as WireGuard VPN. Title: "WireGuard: Next Generation Kernel Network Tunnel" Authors: Jason A. Donenfeld URL: <https://www.wireguard.com/papers/wireguard.pdf>
- [15] Source: Research comparing the performance of ChaCha20 with AES on different platforms. Title: "Comparison of AES and ChaCha20 on FPGA and Hardware" Authors: Kanda and Ryoo URL: <https://www.researchgate.net/publication/312210141>
- [16] Source: Research on ChaCha20's resistance to timing attacks, particularly compared to AES. Title: "Security Analysis of ChaCha20"
Author: Jean-Sébastien Coron URL: <https://www.springer.com/gp/book/9783540324949>
- [17] ChaCha20 in Real-Time Systems. Source: Real-time applications of ChaCha20 for secure communication. Title: "Real-Time Cryptographic Operations with ChaCha20", Authors: M. Kisters and F. S. Y. Law URL: <https://dl.acm.org/doi/10.1145/3360227.3360306>
- [18] Sommerville, I. (2011). Software Engineering (9th ed.). Boston: Addison-Wesley.
- [19] ISO/IEC 9126-1 (2001). Software Engineering – Product Quality – Part 1: Quality Model.
- [20] NIST (National Institute of Standards and Technology). (2020). Security and Privacy Controls for Information Systems and Organizations. Special Publication 800-53.
- [21] Bernstein, D. J. (2008). ChaCha, a variant of Salsa20. International Association for Cryptologic Research (IACR).